【转载】第一讲——从黑盒攻击模型到白盒攻击模型

转载 (https://www.qinless.com/category/reprint) · 2022-04-01 · 1208 次浏览

文章目录[隐藏]

- 转载 18 / 25
- 说说在前面
- 从黑盒攻击模型到白盒攻击模型
- 一、引言
- 二、密码学基本原则
- 三、攻击模型
  - 3-1 黑盒攻击模型
  - 3-2 灰盒攻击模型
  - 3-3 白盒攻击模型
- 四、抵御白盒攻击模型下的攻击
  - 6.1 密码学家小A——攻击者为什么能发现是AES?
  - 6.2 设备信息收集工程师小B——攻击者为什么能用IDA和Frida?
  - 6.3 安全工程师小C——攻击者怎么能分析代码?
- 五、尾声

转载 (https://www.qinless.com/series/zhuanzai) 18 / 25

【转载】unidbg hook 使用大全 (https://www.qinless.com/670)

【转载】ida 基本使用 常用快捷键 (https://www.qinless.com/732)

【转载】Frida和调试器共存的问题 (https://www.qinless.com/937)

【转载】Charles抓包Android最佳流量转发APP (https://www.qinless.com/939)

【转载】安卓自定义代码调用系统函数并编译so记录 (https://www.qinless.com/1020)

【转载】jnitrace 手机卡死黑屏问题解决 (https://www.qinless.com/1169)

【转载】Unidbg SO 逆向入门实战教程一 OASIS (https://www.qinless.com/1173)

返回顶部

【转载】Unidbg SO 逆向入门实战教程二 calculateS (https://www.qinless.com/1177)

【转载】Unidbg SO 逆向入门实战教程三 V2-Sign (https://www.qinless.com/1181)

【转载】Unidbg SO 逆向入门实战教程四 mfw (https://www.qinless.com/1185)

【转载】Unidbg SO 逆向入门实战教程五 qxs (https://www.qinless.com/1188)

【转载】Unidbg SO 逆向入门实战教程六 s (https://www.qinless.com/1192)

【转载】Unidbg SO 逆向入门实战教程七 main (https://www.qinless.com/1194)

【转载】Unidbg SO 逆向入门实战教程八 文件读写 (https://www.qinless.com/1201)

【转载】Unidbg SO 逆向入门实战教程九 blackbox (https://www.qinless.com/1204)

【转载】Unidbg SO 逆向入门实战教程十 simpleSign (https://www.qinless.com/1209)

【转载】android app 加密参数分析研究 hash aes (https://www.qinless.com/1634)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型

【转载】第二讲——白盒加密攻击方法的选择 (https://www.qinless.com/1645)

【转载】第三讲——差分故障攻击的原理 (https://www.qinless.com/1647)

【转载】第四讲——差分故障攻击的工具链 (https://www.qinless.com/1653)

【转载】第五讲——使用源码进行DFA攻击 (https://www.qinless.com/1655)

【转载】第六讲——使用Frida进行DFA攻击 (https://www.qinless.com/1657)

【转载】第七讲——使用Unidbg进行DFA攻击 (https://www.qinless.com/1659)

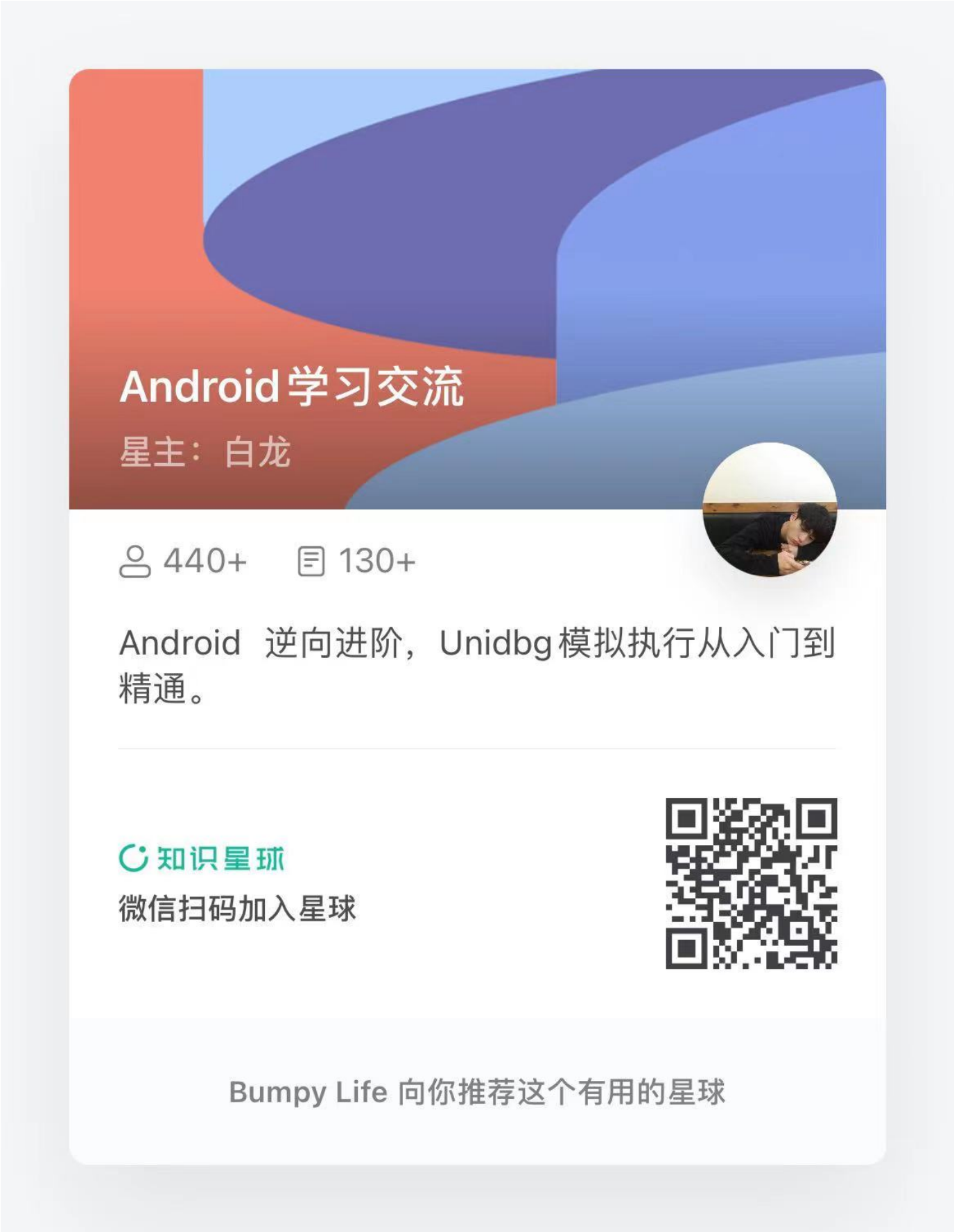
【转载】第八讲——案例 (https://www.qinless.com/1661)

« 上一篇文章 (https://www.qinless.com/1634)
下一篇文章 (https://www.qinless.com/1645) »

说说在前面

本文转载龙哥星球 白盒 AES 密码学系列，相关文件代码等，可加入星球下载

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/03/longge-zsxq.jpeg)

从黑盒攻击模型到白盒攻击模型

返回顶部

一、引言

这是一个系列文章的第一篇，这篇讲白盒加密的产生环境和历史原因。好比琼瑶剧，现在的眼光看起来，有些三观不正。但结合反封建反父权的时代背景，就合理的多了。白盒加密也应该这么去端详。

整个系列粗浅的讲白盒加密的相关概念，以及从白盒加密中提取密钥的相关技术。

好友 *nu11* 以及 *runtime* 给这个系列文章帮了很大忙。

二、密码学基本原则

柯克霍夫原则——**即使密码系统的任何细节都被人知道，只要密钥未泄漏，它就应该是安全的。**这在实践上很有启发性，我们可以找到至少三个佐证

- 保密整个加密算法的难度一定比保密单条密钥更大
- 安全性依赖于加密算法的保密时，一旦算法泄露，采用该种加密算法的所有系统都会受影响；安全性依赖于密钥保密时，一旦密钥泄露，只有使用这个密钥的特定场景受影响。
- 应对泄露，前者需要重新设计加密算法，后者只需要更换密钥。

可是尽管加密的需求以及加密算法的产生已有数千年，但是满足柯克霍夫原则的强有力且容易使用的加密算法并不易得。加密算法不安全的一个著名例子是**《玛丽女王的信件》**。

那是1586 年，中国处于明朝时期，地球另一面的大不列颠岛上，玛丽女王因叛国罪正在服刑。狱中几年，她一直用加密信件和同党联络，密谋刺杀英格兰女王伊丽莎白一世，但这些加密信件往来被人截获，交给了密码学家。通过对信件密文做频率分析，密码学家破解了该种加密，最终玛丽女王被送上了断头台。

安全的加密算法在哪里？这个问题长久没有答案。我们应当意识到，加密的第一需求是战争，在二战中密码学得到了充分的发展。但是，战争结束后，相关的技术并未公开。原因很简单——各国想在下次战争或日常机密保护中继续使用这些密码，公开这些成果让他们感到不舍。

这个时候加密的第二需求——商业，促进了密码学的真正开源和发展。二战技术后，经济复苏，商业繁荣，许多商业场景需要可靠的加密算法，用于数据加密、版权保护等处。在商业需求的呼唤下，美国公开评选出了DES，全称 $Data$ ，$Encryption$ ，$Standard$ ，即数据加密标准。

这个博客部

美国标准很快成为世界标准（除了苏联以外其辐射地域外的世界），DES 成为世上第一个广为流传，源码公开，用于工商政企，且满足柯克霍夫原则的加密算法。好景不长，随着计算机算力发展，DES 的暴力破解难度下降。在21世纪，美国NSA评选出了继任者——AES，$Advanced$，$Encryption$，$Standard$ 即高级加密标准。

在人们使用加密算法的使用中，遇到了一个麻烦的问题——密钥如何保密？如果我们要与他人通信，像玛丽女王那样，那可以把密钥记在心里，遇到加密需求就在心里算一下，但现代加密算法的复杂度很高，无法靠心算解决，而且现代商业模式下，加密需要在硬件或软件上运行，即密钥不仅得保存在你的心中或保险箱里，也写在固件/芯片/二进制文件里，安全隐患随其而来。

如果攻击者可以接触运行加密算法的设备，那么他可以利用高分辨率显微镜，在化学实验室中对芯片做逆向，提取出其上的密钥。如果加密算法运行在PC上，调试器或Hook工具能拦截和获取加密程序的密钥。除此之外，在90年代，研究人员开发了侧信道攻击技术。原理很巧妙，研究人员观测到，加密软件或硬件运行时存在电磁变化、噪音大小、能量消耗、温度变化等现象，但这些信息里竟然暗示甚至直接泄露了执行流中的敏感信息，以至于可以提取出密钥。在此类攻击中，分析者不必拆开设备，只需要监控设备周围的环境信息，就有可能提取出密钥。

攻击方式复杂多样，五花八门，于是研究人员根据加密程序其运行环境的可信赖程度，划分出黑盒/灰盒/白盒攻击模型。需要注意的是，黑/灰/白盒的说法借鉴了软件测试中的同名概念，但定义并不完全相同。比如白盒这一概念，在软件测试中一般指有源代码的情况，但攻击场景中一般不会到有源代码这个地步，所以只要能随意hook/debug/call function/watch memory，就算白盒了。

三、攻击模型

3-1 黑盒攻击模型

在黑盒攻击模型中，运行环境被认为完全安全。举一些例子，比如银行的交易系统、政府的机密系统，或者大公司的内部机房，属于黑盒攻击模型。毕竟攻击者总不能去政府机房里用拆机器吧?

黑盒攻击模型中假定，攻击者无法对**运行过程**做任何的窥探和分析，但一般能获取输入和输出，并按照掌控程度可分如下几类

- 唯密文攻击——攻击者仅获取了若干的密文，不清楚任何的明文信息。比如分析玛丽女王密件的密码学家，他的攻击场景就是唯密文攻击，古典加密算法强度较低，许多知名的**古典加密算法**都可采用唯密文攻击攻克。

返回顶部

- 已知明文攻击——攻击者获取了若干明密文对。
- 选择明文攻击——攻击者可以控制明文输入，并获取对应输出。

黑盒攻击模型下，攻击主要聚焦在对加密算法的密码学分析。如果某种算法可以在黑盒攻击模型下可以被轻易提取密钥，那它一定是不安全、不建议使用的加密算法。学术界对AES算法的黑盒攻击研究已经证明，黑盒攻击只能略微降低AES的安全性，想轻易提取出Key，毫无可能。除此之外，实践中发现，不安全的分组工作模式会带来极大安全隐患，但这并非本文的主题了。

3-2 灰盒攻击模型

灰盒攻击模型比黑盒攻击模型更符合实际中的多数场景。银行的交易系统等毕竟是少数。手机卡、信用卡等各种智能卡也需要加密算法的保护，这是更多数的情况。

在此种情况下，攻击者可以做更多的事，而不是仅仅观察加密的输入与输出。攻击者可以物理接触设备，或者在设备周围安装检测器。

- 侵入式攻击，拆开设备，使用昂贵的高分辨率显微镜和化学实验室，对加密算法所位于的芯片做分析，提取其中的密钥。
- 侧信道攻击，如上文所说，设备在运行加密程序时，会间接泄露敏感信息，这些信息以加密耗时、声音噪声、机器温度、电磁辐射等等形式展示。一个平价的示波器就可以进行测信道攻击。

买个示波器测测自己游戏机的电磁辐射，总没人管吧？这就是灰盒攻击模型。对于一些重要或机密的情况，可以在硬件上会做处理，使得加密耗时、声音噪声、机器温度、电磁辐射等角度的信息泄露尽可能少，以此对抗灰盒攻击。但普通的商用产品，没精力和能力管这些。

如果说黑盒攻击是密码分析学家的主场，那么灰盒攻击则是每个专业人员就可掌握的拿手好戏，买个示波器就能进行基本的灰盒攻击。但必须得承认，灰盒攻击并未“飞入寻常百姓家”，它不是人人都能搞定的玩意。

3-3 白盒攻击模型

大人，时代变了！

返回顶部

(https://www.qinless.com/wp-content/uploads/2022/04/6246fe2bbd2e7.png)

随着PC个人电脑的发展，商业模式大变。应用以代码资源包的形式分发。密码学在这些商业应用上同样必不可少，比如保护通信安全、维护具有版权的数字资料、加密个人数据等等，此时加密算法以软件程序的形式存在。

而分析者拥有对运行环境（电脑或手机）近乎完全的控制权，对付软件实现的加密，**可以静态分析程序的二进制代码，可以观察程序的汇编执行流，可以读写、修改程序的内存。**仅仅需要一台个人设备，再下载几个调试软件，就能实现密钥的提取！

终端设备的改变，让运行环境越来越不可信，攻击成本越来越低，可谓是“旧时王谢堂前燕，飞入寻常百姓家”。任何人找几个学习视频，花上一段时间就能掌握白盒攻击模型下的这些技术，进而对软件应用的安全产生威胁。

在白盒攻击场景下，采用什么加密算法无足轻重，柯克霍夫原则甚至成了帮凶。攻击者从内存中dump出密钥，从执行流中Hook出密钥，再IDA中直接看到密钥。反正怎么舒服怎么来，然后在网上找到应用所使用加密算法的源码，填入密钥，万事大吉。

那么，在白盒攻击场景下，如何做安全保护？这是一个值得深思的问题。

考虑Android中如下场景

返回顶部

分析者使用IDA静态分析SO，发现是AES加密，确认哪一个参数或地址存放密钥后，使用IDA Debug 或 Frida Hook 获取密钥，和标准算法加密结果比对，确认一致，收工。

怎么阻挠如上的攻击？

四、抵御白盒攻击模型下的攻击

首先我们得理解，什么叫抵御白盒攻击模型下的攻击，**对于密码学部分而言，让攻击者没法直接找到Key就结束分析，而是要复写加密流程或者俗称抠代码，这就已经是抵御。**没什么办法能阻挠逐行汇编的复写代码，这是白盒场景自身决定的。

鲁迅评价《红楼梦》：**经学家看见《易》，道学家看见淫，才子看见缠绵，革命家看见排满，流言家看见宫闱秘事。**

同一个问题，不同的人，不同的专业角度，会有不同的解决方案，对待上面所发生的攻击也同理。

6.1 密码学家小A——攻击者为什么能发现是AES？

攻击者所谓 **发现是AES加密** 是怎么做到的？密码学家们发明了成千上万的加密算法，攻击者怎么确认这算法就是AES呢？小A对这一句话很不解，调查后他发现

在现行加密标准中，AES最为权威、官方、通用、高效，因此深得开发者青睐，除此之外的常见算法有DES、RSA、MD5、SHA1等几款。就好比娱乐圈，每个时期称得上顶流的明显不超过十个，各种大场面上都是这么一批人。那么只要记住这几个人的脸，看各种大会总能脸熟。攻击者也是一样，只需要了解和熟悉最常见的几种加密算法，在逆向分析时往往就是它们。

小A开始思考，那能不能不选择顶流呢？非要在AES/DES上吊死吗，但他发现了两个阻碍

- AES/DES这样的算法，得到了全世界密码学专家和分析者的检验与认可，安全性最高。算法都是开源免费的，那肯定用最好的，不是吗？
- 开发者很可能只熟悉AES/DES这样的常见加密算法，一般的课程或培训里，教的也是这些。选其他加密算法，有额外的学习成本，稍有不慎会带来其他安全隐患，而且小众加密算法的开源实现也较少。

而如果最终克服了上述障碍，采用一个不那么”顶流“的加密算法，是不是就能迷惑攻击者，无法识别了呢？这其实有些道理。但可惜的是，早在21世纪初，就有研究人员指出，大部分加密算法在运算流程中都存在固定且已知的常数或表，那么可以收集常见的数十种加密算法的这些”特征“，在二进制文件中进行字节流的检索与匹配，从而识别这些算法，这就是各种逆向分析工具中Findcrypt工具的由来。

返回顶部

感兴趣的可以查看 findcrypt3 (https://github.com/polymorf/findcrypt-yara/blob/master/findcrypt3.rules) 的规则文件，了解这些常数和表。相对完善的 Findcrypt 包括但不限于如下加密算法。

返回顶部

COPY

```
SKIPJACK
Tiger
Twofish
Blowfish
DES
CAST5
RC2
RC5
RC6
NOEKEON
SEED
CAMELLIA
SOBER128
MARS
SHARK
Square
WAKE
Sosemanuk
SHACAL2
Kalyna
ARIA
EDONR
KASUMI
KHAZAD
SAFER
GOST
MISTY1
SM4

MISC:
CRC24
CRC32
Base64
Base64url
zlib
lua51
lua52
lua53

Hash:
MD2
MD4
MD5
```

返回顶部

```
SHA-1
SHA-224
SHA-256
SHA-384
SHA-512
Whirlpool
BLAKE2B
BLAKE2S
SHA-3
HAVAL
SNEFRU
SM3
STREEBOG
Bcrypt
```

如果说，原先攻击者只对顶流脸熟，那么 $Findcrypt$ 就是一个贴心的秘书，每天关注数十个当红流量小生。当攻击者分析算法缺少线索，甚至一无所获时，就叫来秘书帮看看。如果属于秘书了解的几十个加密算法之一，攻击者就专门学习和了解它，进而分析密钥，进行验证。那么 **发现是AES加密** 就变成了 **发现是ABC加密或EFG加密**，防护作用并不大。

当然，还存在更多 $Findcrypt$ 也不知道的四五六线明星，但这样知名度较低的算法，安全性能保证吗？你用的放心吗？领导同意上线吗？

那么不如换个思路，不用已知的加密算法。即自研一种加密算法，这样攻击者就没法获取密钥就获取一切，而是得还原整个加密算法。首先，这存在和上述一样的问题——如何保证它的安全性。知名度较低的加密算法，我们都忧虑其安全性，更何况公司内部自研的加密算法呢？因此这个思路也有问题。实际分析场景中，确实会看到一些大公司自研的算法。但往往也只用于加密的某一环节，逻辑主体仍然依赖常见的加密算法。

小A又想，那能不能步子跨的小一些，在标准算法上进行适当魔改？这样效果上类似于自研算法，攻击者得还原整个魔改算法，安全性上又类似原算法，因为只是做了些改动。

如果攻击者十分熟悉你所魔改的那一种加密算法，那他必须在仔细对比流程和步骤后，找出魔改和标准版的差异之处，修改后才能顺利使用。如果分析者不那么熟悉那种加密算法，那就只能看待为一种新算法，逐步骤复写加密流程，实现函数功能。不管是哪一种，对攻击者来说都是不小的开销，一定程度上阻碍了分析。

返回顶部

因此尽管魔改加密算法存在如下一些缺点，但依然是个好方案。

- 不恰当的魔改可能会削弱安全性，但一定比自研加密算法安全性高
- 魔改算法不利于和其他业务互通逻辑，你这处是魔改AES，别处都是标准AES，互通有障碍。

在实践中我们发现，许多程序选用了这一办法。常见魔改算法有 AES/RSA/MD5/SHA1。根据操刀者的水平以及各方考量，魔改的点也不同。轻微的改常数，深度的变流程，其中的骚操作非常多。

有没有更好的方案呢?

白盒密码就是一个更好的方案，或者至少是一个不那么差的方案。其核心思想是把密钥和加密流程融为一体，实现密钥的隐藏。

我们不妨看一下白盒加密的提出者——Chow，在其2002年论文中的阐述

如果使用者可以控制执行环境，或者执行环境中存在恶意软件，加密算法的传统软件实现是完全不安全的。然而，目前的趋势是，这种“不安全的运行环境”的使用率在不断增加。我们讨论了一种加密函数的方案，旨在这种不可信任的运行环境中，提供一种可抵御白盒攻击的、有实用价值的保护。

这是Chow 在2002年的推断，当时所担忧的“不安全的执行环境”是PC电脑，Chow判断，其使用会不断增加，可以说很有远见。如今这种**不安全的执行环境** 还包括移动端手机，IOS以及Android。

相较于魔改加密算法的方案，白盒加密不会削弱安全性，因为输入输出和标准加密完全一致，而且因为和标准加密一致，所以不会影响不同业务的互通。因为无法找到密钥，所以攻击者只能把白盒加密整体流程逐步骤复写下来，才能执行调用。因此可以说，白盒加密对业务和安全性的影响比魔改算法小，对白盒攻击模型下攻击的抵抗程度又至少和魔改加密一致，显然是一个好方法。

这就是Chow 在2002年的论文中提出的白盒加密其需求背景：即使运行环境被攻击者完全掌控（比如白盒攻击模型），加密算法的密钥也不应该轻易泄露。为了实现这一目标，他提出可以将密钥融入加密程序，使之成为一体，这个方案就叫白盒加密，并给出了白盒AES的实现。

因此从密码学视角的保护方案上看，白盒密码、魔改加密算法、自研加密算法同属一类，白盒加密属于其中较好的一种方案。

返回顶部

6.2 设备信息收集工程师小B——攻击者为什么能用IDA和Frida?

所谓白盒攻击环境，指的是攻击者拥有”**可以静态分析程序的二进制代码，可以观察程序的汇编执行流，可以读写、修改程序的内存**“的几乎完全的掌控能力。

小B认为，**君子性非异也，善假于物也。**灰盒攻击中，示波器、显微镜等是必要的工具。好比一个游戏机里用了AES加密，配合示波器可以做灰盒攻击，最终分析出密钥。那如果对示波器设备做管控，禁止购买呢？攻击者没了示波器，就没法利用电磁辐射做侧信道攻击，进而阻挠了灰盒攻击。这是一个很可靠的思路，小B发现，白盒攻击中，也有其设备和工具要求。

以工具为例：OllyDBG、IDA、Frida、Xposed、Substrate等，就是各种平台上，常见的、用于白盒攻击模型下的攻击工具。

以设备为例：PC上的虚拟机，Android上的Root设备，IOS上的越狱设备，则是其常见或必要条件。

接下来主要分析Android场景，在Android上，一台Root设备是标配，除此之外，为了增加功能，便于魔改系统等，部分攻击者的测试机是Pixel这类国内几乎没正常用户会用的Google手机。

一般用户的设备很少做Root，那么可以进行检测，当设备已Root时，判断为存在风险的设备，走入不同的运算分支/拒绝服务/马上出警等，都是好的应对方案。常见的Root检测有功能检测/文件检测/相关文件检测等。许多对安全性要求高的政企类App确实在这么做，比如银行应用——很多都对Root设备说不。攻击者在设备这一关就过不去了，自然称不上什么白盒环境，连黑盒都不算！黑盒好歹还提供还可以观察加密的输入和输出，但我们可以做到停止服务！

退一步讲，如果分析者绕过了Root检测，他也并非就直接获得了白盒攻击的能力。Frida他得用吧？IDA得用吧？不用Frida那别的插桩工具得用吧？不用插桩工具那得修改SO文件本身吧？这里面有许多方法和思路可以做阻挠。

以IDA为例，Android逆向SO时，需要IDA进行动静态分析。

加壳/字符串加密/间接跳转/代码加密等技术可以阻挠静态分析，在老版IDA上，由于SO加载器不完善，甚至可以构造无法被IDA直接打开的SO；花指令等技术可以阻挠IDA F5；检测调试器、检测端口、检测文件等技术可以阻挠IDA动态调试。一个无法顺利使用IDA的攻击者，其白盒攻击的能力一定打了折扣。或者至少，这些技术可以增加攻击者的分析成本。攻防的本质是成本对抗，成本越高，攻击的性价比越低。

返回顶部

再以Frida这个热门Hook工具为例，Frida 注入和Hook的痕迹可不少，Maps检测/文件检测/端口检测/Inline Hook检测/内存搜索等，可使用的检测层出不穷。如果既无法使用Frida，又无法使用IDA，许多人就得抓耳挠腮了。

除此之外，测试机还有许多其他特征。比如测试机往往装的App较少，国民应用也可能没装。一个手机没下载淘宝/支付宝/微信合理吗？这不合理啊，因此可以从侧面辅助推断是否是风险机。

有些攻击者爱用模拟器/云手机/设备多开，同样有大量特征可检测。

除此之外，还有大量的、相关的特征，比如是否魔改了系统，比如脱壳机系统，比如待机时长/网络状况/电池情况/传感器数据/应用安装时间/关联应用等等。

因此小B认真研究了相关特征，做了一个SDK，检测白盒攻击场景下的常见的工具。只要制止攻击者使用相关工具和设备，白盒场景就会降级成灰盒攻击场景，那么 “**白盒场景下，密钥容易被攻击者直接获取**” 就不是我们要面对的问题了。

6.3 安全工程师小C——攻击者怎么能分析代码？

小C是一名安全工程师，他精于Ollvm，VMP等代码混淆技术。开发者写了一千行的代码，他可以混淆成了十万行，汇编执行流更是达到数千万行。其中有效逻辑只占整体的百分之一，完全可称为屎山。在小C看来，这么一套下来，攻击者可能分析一周的指令，都没找到加密的入口呢，更别提分析什么密钥了！

同时从小C视角看，白盒加密技术是一种局限于加密算法程序的混淆技术。

首先看混淆的定义：**混淆让信息以一种完全无法理解的形式存在，尽量让人无法理解中间的过程（也就是只能看到输入和输出，但无法理解结果是如何得到的），但不影响信息本身发挥作用。**

一个OLLVM后的程序，尽管逻辑变成复杂和稀碎，但功能实现完全没有变化。一个白盒化的加密算法，密钥正常发挥作用，但无法直接找到密钥。**因此，白盒加密可以理解为基于密码学，且作用于密码程序的特定混淆技术。**

反过来说，白盒加密的目的是隐藏密钥，那么，一个标准实现的AES，采用Ollvm等技术混淆代码成几千行，这不也一定程度上阻碍了对密钥的分析了吗？是否也算一种白盒密码呢？

我认为广义上算，但是，因为程序混淆和白盒加密是各自独立发展出来的概念，所以白盒加密一般独指基于密码学的、隐藏密钥的技术。

返回顶部

值得一提的是，Chow 提出白盒密码的2002年，程序混淆技术还不成熟，他在论文中也表示，对混淆技术的能力和效果持观望态度。可以大胆猜测，如果那时的程序混淆保护技术就已然成熟，那么白盒加密很可能不会引起较大反响，甚至不会出现。因为白盒加密只是对加密程序的保护，而程序混淆技术是对白盒场景下全体程序的保护，更通用也更有效。而且事实上，白盒加密的安全性也一直没得到业界认可。

五、尾声

上文中，我们从一个尽量宽广的视角来看待白盒加密，希望对大家理解白盒加密的出现有所帮助。在实践中我们发现，随着技术的发展和迭代，小A，小B，小C的思路和方案都有道理，但又都不全面。

只听小A是不行的，清晰的加密代码逻辑，不管是抠代码、RPC还是做分析都很方便。

只听小B也不行，小B试图阻止白盒攻击所依赖的设备和工具，这是很有意义的，但我们知道，只要付出足够的努力，检测都可以过掉，白盒攻击环境终会成立。

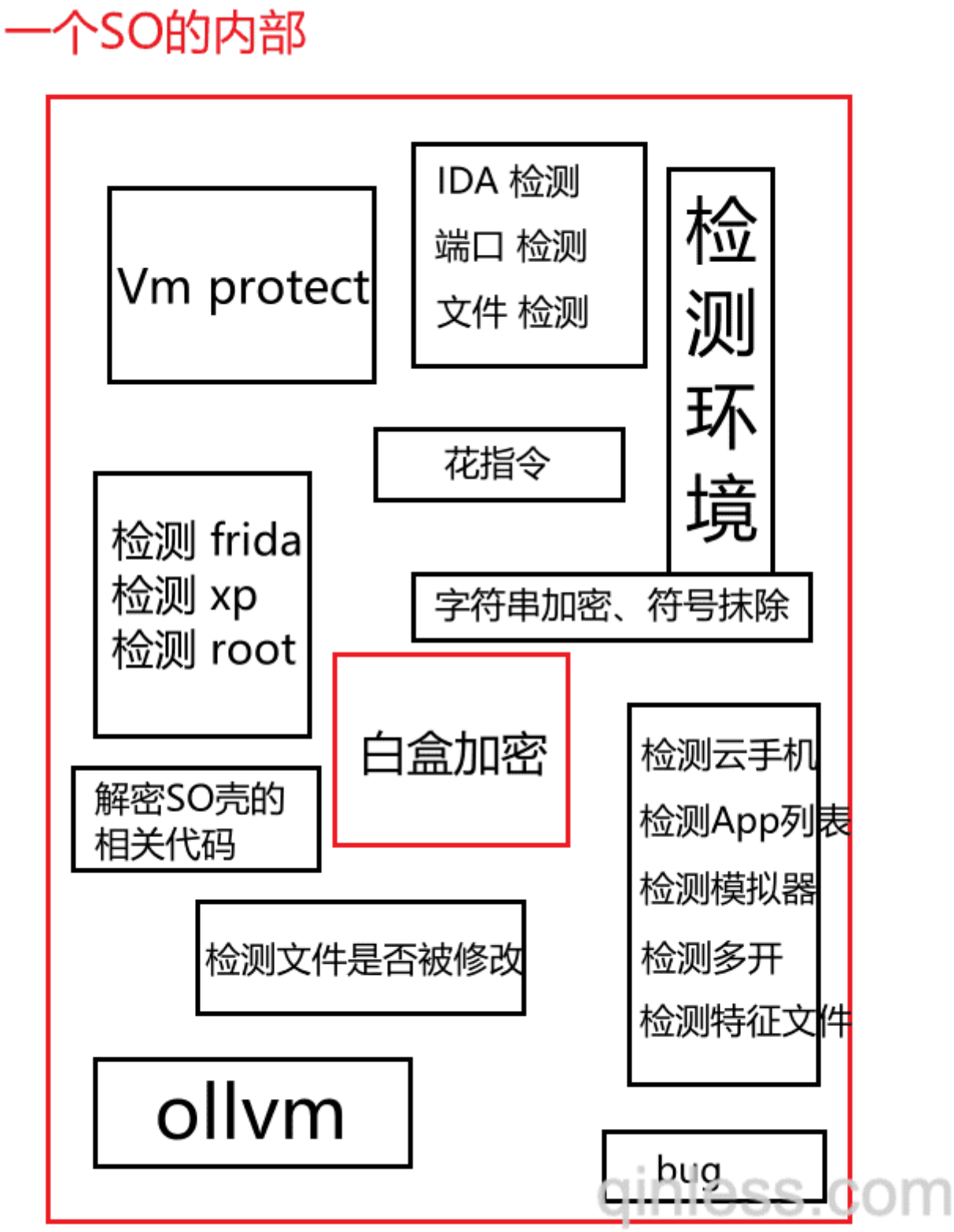
小C的程序混淆十分有用，但并非所有的场景都适合强混淆，也得考虑效率和性能。Android的算力和存储在提升，但远没有到能随意挥霍的地步。举个例子，如果一个安全SDK占500M空间，那么不管它有多安全，都不太可能在常规APK中原生使用。因为ApK稍大一些，很多用户就会不愿意下载。为了安全性牺牲下载量和用户，这是绝大多数厂商都无法忍受的事。

因此在事实上，小A、小B、小C以及无数小X的思路被取长补短、均衡有序、适当取舍的采纳了，你可能会在单个SO里见到上面几种方案。

一个样本可能加了壳，做了大量检测和对抗，以期避免被白盒攻击。而且SO还做了充足的混淆，乱成一团的的汇编，F5也没法成功，IDA trace完整一次要好几十个小时，几十个G的 traceCode文件。然后其中的某一小段，是一个白盒AES。

你真的扛得住吗?

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/6246fe372c4c9.png)

这个系列主要聚焦于白盒加密中密钥的提取，但所分析的样本中有其他元素时，也会予以讨论。

标签： # aes，# dfa，# 攻击模型，# 白盒aes (https://www.qinless.com/tag/baiheaes)

返回顶部

(https://www.qinless.com/1642)

(https://www.qinless.com/wp-content/themes/honey/public/qrcode?data=https://www.qinless.com/1642)

(https://www.qinless.com/1642) 暂无评论 (https://www.qinless.com/1642#respond)

分享一个 android studio 分析 apk 反编译代码的技巧

上一篇

(https://www.qinless.com/1640)

【转载】第二讲——白盒加密攻击方法的选择

下一篇

(https://www.qinless.com/1645)

会爬山的小脑虎 (https://www.qinless.com/author/xiayu)

小爬爬

本文作者： **会爬山的小脑虎 (https://www.qinless.com/author/xiayu)**

本文链接： **https://www.qinless.com/?p=1642 (https://www.qinless.com/1642)**

版权声明：本博客所有文章除特别声明外，均采用 CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/) 许可协议。转载请注明来自 qinless 的博客！ (https://www.qinless.com)

微信搜一搜

逆向小友

发表评论

返回顶部

昵称 *

邮箱 *

网址

输入评论内容...

提交评论

Copyright 2022. All Rights Reserved. Powered By XinTheme (http://www.xintheme.com) + WordPress 果酱 (https://blog.wpjam.com/) 浙ICP备2021029793号 (http://www.miibeian.gov.cn/)

返回顶部